```
/*
 * ====================================================================
 * ORION - ESP32-S3 + MECANUM + GARRA + CACAMBA
 * CODIGO UNIFICADO V4.1.2 - MANUAL + TURBO RT + AUTONOMO APRENDIDO
 *
 * Arduino-ESP32 Core 2.x + Bluepad32
 * IMPORTANTE: manter/fixar o Arduino-ESP32 Core 2.x neste firmware.
 *
 * CONTROLES
 * - Analogico esquerdo Y: frente / tras
 * - Analogico esquerdo X: lateral
 * - Analogico direito X : giro
 * - RT                  : limita a saida final de 70% ate 100%
 * - A segurado          : garra ativa imediatamente
 * - Y segurado          : cacamba sobe suavemente
 * - L1 + R1 por 3 s     : arma aprendizado; solte e centralize para iniciar
 * - L1 + R1 por 3 s     : durante aprendizado, finaliza e salva
 * - B durante preparacao/aprendizado: cancela sem substituir a macro salva
 * - R1 + D-Pad          : executa macro aprendida (1 vez por aperto)
 * - B no autonomo       : cancela a execucao
 *
 * AUTONOMO
 * - Ate 30 segundos.
 * - Continua mesmo se o controle desconectar.
 * - Ao terminar, para imediatamente.
 * - Macro fica salva na memoria mesmo depois de desligar a ESP32-S3.
 *
 * SEGURANCA
 * - No modo manual, 300 ms sem dados = motores param.
 * - Se perder controle durante aprendizado, aborta e preserva a macro antiga.
 * - Ao salvar, motores e cacamba ficam congelados ate concluir a gravacao.
 * - Retomadas manuais exigem analogicos no neutro.
 * - Uma nova gravacao so comeca depois de soltar L1+R1 e centralizar os analogicos.
 * - Falha/desconexao: rodas param e servos ficam na ultima posicao comandada.
 * - Garra e cacamba sao independentes e podem mover ao mesmo tempo.
 * ====================================================================
 */
#include <Bluepad32.h>
#include <Preferences.h>
#include "driver/mcpwm.h"
#include <math.h>
#include <string.h>
#include <stddef.h>
// ====================================================================
// SENTIDO DAS RODAS
// ====================================================================
constexpr bool INVERTER_SENTIDO_GERAL = false;
constexpr int8_t SINAL_FE = 1;
constexpr int8_t SINAL_TE = 1;
constexpr int8_t SINAL_FD = 1;
constexpr int8_t SINAL_TD = 1;
// ====================================================================
// PINOS
// ====================================================================
// Frente esquerda
constexpr uint8_t PIN_FE_A = 7;
constexpr uint8_t PIN_FE_B = 15;
// Traseira esquerda
constexpr uint8_t PIN_TE_A = 5;
constexpr uint8_t PIN_TE_B = 6;
// Frente direita
constexpr uint8_t PIN_FD_A = 18;
constexpr uint8_t PIN_FD_B = 3; // GPIO3 e strapping pin na ESP32-S3
// Traseira direita
constexpr uint8_t PIN_TD_A = 13;
constexpr uint8_t PIN_TD_B = 4;
// Servos
constexpr uint8_t PIN_SERVO_GARRA   = 16;
constexpr uint8_t PIN_SERVO_CACAMBA = 17;
// ====================================================================
// CANAIS LEDC DOS MOTORES
// ====================================================================
constexpr uint8_t CH_FE_A = 0;
constexpr uint8_t CH_FE_B = 1;
constexpr uint8_t CH_TE_A = 2;
constexpr uint8_t CH_TE_B = 3;
constexpr uint8_t CH_FD_A = 4;
constexpr uint8_t CH_FD_B = 5;
constexpr uint8_t CH_TD_A = 6;
constexpr uint8_t CH_TD_B = 7;
// ====================================================================
// MOTORES / JOYSTICKS
// ====================================================================
constexpr uint32_t PWM_FREQ = 1000;
constexpr uint8_t PWM_BITS = 8;
constexpr int PWM_MAX       = 255;
constexpr int DEADZONE = 45;
// Equivale aproximadamente a SUAVIZACAO 0.30 com loop de 15 ms,
// mas agora a resposta depende do tempo real transcorrido.
constexpr float TAU_MOVIMENTO_MS = 42.0f;
// RT limita a saida FINAL das rodas: 70% solto -> 100% pressionado.
constexpr float VELOCIDADE_NORMAL = 0.70f;
constexpr float VELOCIDADE_TURBO = 1.00f;
constexpr uint8_t ESCALA_NORMAL = (uint8_t)(PWM_MAX * VELOCIDADE_NORMAL + 0.5f);
constexpr uint8_t ESCALA_TURBO = PWM_MAX;
// 0 = desativado.
// Se os motores tiverem uma zona morta, aumente depois de medir na bancada.
constexpr int PWM_MINIMO = 0;
constexpr uint32_t FAILSAFE_MS = 300;
constexpr uint32_t LOOP_MS = 15;
// Deixe true durante testes de bancada.
// Na competicao, pode mudar para false para silenciar a telemetria periodica.
constexpr bool DEBUG_SERIAL = true;
// ====================================================================
// GARRA E CACAMBA
// ====================================================================
constexpr uint8_t ANGULO_GARRA_REPOUSO = 0;
constexpr uint8_t ANGULO_GARRA_ATIVA   = 60;
constexpr uint8_t ANGULO_CACAMBA_REPOUSO = 0;
constexpr uint8_t ANGULO_CACAMBA_ATIVA   = 60;
// Garra: imediata.
// Cacamba: progressiva por tempo, ~mesma velocidade da versao anterior.
constexpr float CACAMBA_GRAUS_POR_SEG = 66.7f;
constexpr uint16_t SERVO_MIN_US = 500;
constexpr uint16_t SERVO_MAX_US = 2400;
constexpr uint16_t SERVO_FREQ   = 50;
// ====================================================================
// BLUETOOTH / APRENDIZADO / AUTONOMO
// ====================================================================
constexpr bool LIMPAR_PAREAMENTO_NO_BOOT = false;
// L1 + R1 segurados por 3 segundos.
constexpr uint32_t TEMPO_COMBO_APRENDER_MS = 3000;
// Regra da prova: no maximo 30 segundos de autonomo.
constexpr uint32_t TEMPO_MAX_AUTONOMO_MS = 30000;
// Evita gravar tremidas pequenas do joystick.
constexpr int DELTA_GRAVACAO = 18;
// Pequenas variacoes analogicas do RT tambem sao filtradas.
constexpr int DELTA_RT_GRAVACAO = 4;
// Segmentos analogicos muito curtos sao filtrados.
// Eventos criticos (parada, inversao, servo e flags) podem ser menores.
constexpr uint32_t SEGMENTO_MINIMO_MS = 75;
// Limite de eventos gravados. Se atingir antes de 30 s, salva o que coube.
constexpr uint16_t MAX_PASSOS_MACRO = 420;
// ====================================================================
// BLUEPAD32
// ====================================================================
ControllerPtr meuControle = nullptr;
// ====================================================================
// MODOS E ESTADO DO ROBO
// ====================================================================
enum ControlMode {
    PROCURANDO,
    CONTROLE,
    APRENDENDO,
    AUTONOMO
};
ControlMode modoAtual = PROCURANDO;
struct RobotState {
    // Movimento atual suavizado
    float vx = 0;
    float vy = 0;
    float vr = 0;
     // Movimento desejado (sempre em escala completa -255..255)
     int16_t targetVx = 0;
     int16_t targetVy = 0;
     int16_t targetVr = 0;
     // Limite final de saida aplicado DEPOIS da cinematica/normalizacao.
     uint8_t escalaSaida = ESCALA_NORMAL;
     // Garra
     bool garraAtiva = false;
     // Cacamba. A posicao e float para a velocidade ficar independente do loop.
     float cacambaAtual = ANGULO_CACAMBA_REPOUSO;
     uint8_t cacambaTarget = ANGULO_CACAMBA_REPOUSO;
     uint32_t lastCacambaUpdate = 0;
     // Suavizacao dos motores baseada no tempo real.
     uint32_t lastMovimentoUpdate = 0;
     // Ultimo pacote recebido do controle
     uint32_t ultimoDado = 0;
     bool failsafeAtivo = false;
     // Reproducao do autonomo
     uint32_t stepStartTime = 0;
     uint32_t inicioAutonomo = 0;
     uint16_t currentStep = 0;
     uint16_t passoAplicado = 0xFFFF;
};
RobotState robot;
// ====================================================================
// ESTRUTURA DA MACRO
// ====================================================================
enum FlagsMacro : uint8_t {
    FLAG_NENHUMA          = 0,
    FLAG_PARADA_IMEDIATA = 1 << 0,
    FLAG_CONGELAR_CACAMBA = 1 << 1
};
constexpr uint8_t FLAGS_MACRO_VALIDAS =
    FLAG_PARADA_IMEDIATA | FLAG_CONGELAR_CACAMBA;
struct __attribute__((packed)) MacroStep {
    uint16_t duration; // ate 30.000 ms
    int16_t vx;
    int16_t vy;
    int16_t vr;
    int8_t garra;
    int8_t cacamba;
    uint8_t escala;     // RT gravado: 70%..100% da saida final
    uint8_t flags;      // eventos especiais aplicados ao entrar no passo
};
static_assert(sizeof(MacroStep) == 12, "MacroStep deve ocupar 12 bytes");
MacroStep macroAtiva[MAX_PASSOS_MACRO];
uint16_t macroAtivaCount = 0;
MacroStep macroGravacao[MAX_PASSOS_MACRO];
uint16_t macroGravacaoCount = 0;
MacroStep estadoSegmento;
bool segmentoAtivo = false;
uint32_t inicioSegmento = 0;
uint32_t inicioAprendizado = 0;
uint32_t tempoMacroGravado = 0;
uint32_t inicioComboAprender = 0;
bool comboAprenderDisparado = false;
bool aguardandoSoltarParaGravar = false;
bool aguardandoNeutroParaIniciarGravacao = false;
bool pausaComboAprendizadoAtiva = false;
// Toda retomada do manual exige analogicos no centro.
bool aguardandoNeutroParaDirigir = true;
// R1 + D-Pad dispara uma unica vez ate que o combo seja solto.
bool macroDisparoTravado = false;
// Orientacao da reproducao da macro.
enum OrientacaoMacro {
    NORMAL,
    ESPELHAR_DIREITA,
    INVERTER,
    ESPELHAR_ESQUERDA
};
OrientacaoMacro orientacaoAtual = NORMAL;
// ====================================================================
// MEMORIA PERSISTENTE - DOIS SLOTS + CHECKSUM
// ====================================================================
Preferences preferences;
bool memoriaDisponivel = false;
constexpr uint32_t MACRO_MAGIC = 0x4F52494F;   // "ORIO"
constexpr uint16_t MACRO_VERSION = 5;
struct __attribute__((packed)) MacroSlot {
    uint32_t magic;
    uint16_t version;
    uint16_t count;
    uint32_t checksum;
    MacroStep passos[MAX_PASSOS_MACRO];
};
// Um unico buffer temporario evita duplicar memoria da macro.
MacroSlot macroSlotScratch;
constexpr size_t MACRO_HEADER_SIZE = offsetof(MacroSlot, passos);
size_t tamanhoSlot(uint16_t quantidade) {
    return MACRO_HEADER_SIZE + quantidade * sizeof(MacroStep);
}
uint32_t fnv1aAdicionar(uint32_t hash, const void* dados, size_t tamanho) {
    const uint8_t* p = (const uint8_t*)dados;
     for (size_t i = 0; i < tamanho; i++) {
         hash ^= p[i];
         hash *= 16777619UL;
     }
     return hash;
}
uint32_t calcularChecksum(const MacroSlot& slot) {
    uint32_t hash = 2166136261UL;
     hash = fnv1aAdicionar(hash, &slot.magic, sizeof(slot.magic));
     hash = fnv1aAdicionar(hash, &slot.version, sizeof(slot.version));
     hash = fnv1aAdicionar(hash, &slot.count, sizeof(slot.count));
     hash = fnv1aAdicionar(
         hash,
         slot.passos,
         slot.count * sizeof(MacroStep)
     );
     return hash;
}
bool dadosMacroValidos(const MacroStep* passos, uint16_t quantidade) {
    if (quantidade == 0 || quantidade > MAX_PASSOS_MACRO) {
        return false;
    }
     uint32_t total = 0;
     for (uint16_t i = 0; i < quantidade; i++) {
         const MacroStep& p = passos[i];
         if (p.duration == 0) return false;
         if (p.duration > TEMPO_MAX_AUTONOMO_MS) return false;
         if (p.vx < -PWM_MAX || p.vx > PWM_MAX) return false;
         if (p.vy < -PWM_MAX || p.vy > PWM_MAX) return false;
         if (p.vr < -PWM_MAX || p.vr > PWM_MAX) return false;
         if (p.garra != ANGULO_GARRA_REPOUSO && p.garra != ANGULO_GARRA_ATIVA) {
             return false;
         }
         if (p.cacamba < ANGULO_CACAMBA_REPOUSO || p.cacamba > ANGULO_CACAMBA_ATIVA) {
             return false;
         }
         if (p.escala < ESCALA_NORMAL || p.escala > ESCALA_TURBO) return false;
```

```
        if ((p.flags & ~FLAGS_MACRO_VALIDAS) != 0) return false;
        if (p.flags & FLAG_PARADA_IMEDIATA) {
            if (p.vx != 0 || p.vy != 0 || p.vr != 0) return false;
        }
        if (total > TEMPO_MAX_AUTONOMO_MS - p.duration) {
            return false;
        }
        total += p.duration;
    }
    return total <= TEMPO_MAX_AUTONOMO_MS;
}
bool validarSlot(const MacroSlot& slot, size_t tamanhoRecebido) {
    if (slot.magic != MACRO_MAGIC) return false;
    if (slot.version != MACRO_VERSION) return false;
    if (slot.count == 0 || slot.count > MAX_PASSOS_MACRO) return false;
    if (tamanhoRecebido != tamanhoSlot(slot.count)) return false;
    if (!dadosMacroValidos(slot.passos, slot.count)) return false;
    return calcularChecksum(slot) == slot.checksum;
}
bool carregarSlot(uint8_t slotNumero) {
    if (!memoriaDisponivel) return false;
    const char* chave = slotNumero == 0 ? "slotA" : "slotB";
    size_t tamanho = preferences.getBytesLength(chave);
    if (tamanho < MACRO_HEADER_SIZE || tamanho > sizeof(MacroSlot)) {
        return false;
    }
    memset(&macroSlotScratch, 0, sizeof(macroSlotScratch));
    size_t recebido = preferences.getBytes(
        chave,
        &macroSlotScratch,
        tamanho
    );
    if (recebido != tamanho) return false;
    if (!validarSlot(macroSlotScratch, recebido)) return false;
    memcpy(
        macroAtiva,
        macroSlotScratch.passos,
        macroSlotScratch.count * sizeof(MacroStep)
    );
    macroAtivaCount = macroSlotScratch.count;
    return true;
}
bool salvarMacroPersistente(const MacroStep* passos, uint16_t quantidade) {
    if (!memoriaDisponivel) {
        Serial.println(">> ERRO: memoria NVS indisponivel.");
        return false;
    }
    if (!dadosMacroValidos(passos, quantidade)) {
        Serial.println(">> ERRO: macro invalida, corrompida ou maior que 30 s.");
        return false;
    }
    uint8_t slotAtual = preferences.getUChar("active", 0);
    uint8_t novoSlot = slotAtual == 0 ? 1 : 0;
    const char* chave = novoSlot == 0 ? "slotA" : "slotB";
    memset(&macroSlotScratch, 0, sizeof(macroSlotScratch));
    macroSlotScratch.magic = MACRO_MAGIC;
    macroSlotScratch.version = MACRO_VERSION;
    macroSlotScratch.count = quantidade;
    memcpy(
        macroSlotScratch.passos,
        passos,
        quantidade * sizeof(MacroStep)
    );
    macroSlotScratch.checksum = calcularChecksum(macroSlotScratch);
    size_t tamanho = tamanhoSlot(quantidade);
    // Grava somente os passos usados, economizando NVS.
    size_t gravado = preferences.putBytes(
        chave,
        &macroSlotScratch,
        tamanho
    );
    if (gravado != tamanho) {
        Serial.println(">> ERRO: nao foi possivel salvar a macro.");
        return false;
    }
    memset(&macroSlotScratch, 0, sizeof(macroSlotScratch));
    size_t recebido = preferences.getBytes(
        chave,
        &macroSlotScratch,
        tamanho
    );
    if (
        recebido != tamanho ||
        !validarSlot(macroSlotScratch, recebido)
    ) {
        Serial.println(">> ERRO: macro salva falhou na verificacao.");
        return false;
    }
    // So troca o slot ativo depois que a nova copia foi validada.
    if (preferences.putUChar("active", novoSlot) != 1) {
        Serial.println(">> ERRO: macro gravada, mas falhou ao ativar o novo slot.");
        return false;
    }
    return true;
}
void carregarMacroInicial() {
    macroAtivaCount = 0;
    if (!memoriaDisponivel) {
        Serial.println(">> NVS indisponivel. Autonomo aguarda novo aprendizado.");
        return;
    }
    uint8_t slotPreferido = preferences.getUChar("active", 0);
    if (carregarSlot(slotPreferido)) {
        Serial.printf(">> Macro aprendida carregada: %u passos\n", macroAtivaCount);
        return;
    }
    uint8_t outroSlot = slotPreferido == 0 ? 1 : 0;
    if (carregarSlot(outroSlot)) {
        if (preferences.putUChar("active", outroSlot) != 1) {
            Serial.println(">> AVISO: macro reserva carregada, mas nao foi possivel
                marca-la como ativa.");
        }
        Serial.printf(">> Macro reserva recuperada: %u passos\n", macroAtivaCount);
        return;
    }
    Serial.println(">> Nenhuma macro valida. Robo aguardando aprendizado.");
}
// ============================================================
// SERVOS - MCPWM
// ============================================================
void configurarServos() {
    mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM0A, PIN_SERVO_GARRA);
    mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM0B, PIN_SERVO_CACAMBA);
    mcpwm_config_t config = {};
    config.frequency = SERVO_FREQ;
    config.cmpr_a = 0;
    config.cmpr_b = 0;
    config.counter_mode = MCPWM_UP_COUNTER;
    config.duty_mode = MCPWM_DUTY_MODE_0;
    mcpwm_init(MCPWM_UNIT_0, MCPWM_TIMER_0, &config);
}
uint16_t anguloParaPulso(uint8_t angulo) {
    angulo = constrain(angulo, 0, 180);
    return map(angulo, 0, 180, SERVO_MIN_US, SERVO_MAX_US);
}
void escreverServoGarra(uint8_t angulo) {
    mcpwm_set_duty_in_us(
        MCPWM_UNIT_0,
        MCPWM_TIMER_0,
        MCPWM_OPR_A,
        anguloParaPulso(angulo)
    );
}
void escreverServoCacamba(uint8_t angulo) {
    mcpwm_set_duty_in_us(
        MCPWM_UNIT_0,
        MCPWM_TIMER_0,
        MCPWM_OPR_B,
        anguloParaPulso(angulo)
    );
}
// Garra sem rampa: A segurado = ativa, A solto = repouso.
void atualizarGarra(bool ativa) {
    if (ativa == robot.garraAtiva) return;
    robot.garraAtiva = ativa;
    escreverServoGarra(
        ativa ? ANGULO_GARRA_ATIVA : ANGULO_GARRA_REPOUSO
    );
}
// Cacamba suave e independente da garra.
// A velocidade usa o tempo real: atrasos no loop nao mudam a rampa.
void atualizarCacamba() {
    uint32_t agora = millis();
    uint32_t dtMs = agora - robot.lastCacambaUpdate;
    if (dtMs == 0) return;
    robot.lastCacambaUpdate = agora;
    float passo = CACAMBA_GRAUS_POR_SEG * (dtMs / 1000.0f);
    if (robot.cacambaAtual < robot.cacambaTarget) {
        robot.cacambaAtual = fminf(
            robot.cacambaAtual + passo,
            (float)robot.cacambaTarget
        );
    }
    else if (robot.cacambaAtual > robot.cacambaTarget) {
        robot.cacambaAtual = fmaxf(
            robot.cacambaAtual - passo,
            (float)robot.cacambaTarget
        );
    }
    escreverServoCacamba((uint8_t)lroundf(robot.cacambaAtual));
}
// ============================================================
// MOTORES
// ============================================================
int aplicarPwmMinimo(int velocidade) {
    if (velocidade == 0 || PWM_MINIMO <= 0) {
        return velocidade;
    }
    int sinal = velocidade > 0 ? 1 : -1;
    int limite = constrain((int)robot.escalaSaida, 1, PWM_MAX);
    int minimoEfetivo = min(PWM_MINIMO, limite);
    int magnitude = constrain(abs(velocidade), 1, limite);
    // O PWM minimo respeita o teto atual do RT.
    magnitude = map(
        magnitude,
        1,
        limite,
        minimoEfetivo,
        limite
    );
    magnitude = constrain(magnitude, 0, limite);
    return sinal * magnitude;
}
void acionarMotor(
    uint8_t canalA,
    uint8_t canalB,
    int16_t velocidade
) {
    int limite = constrain((int)robot.escalaSaida, 1, PWM_MAX);
    velocidade = constrain(velocidade, -limite, limite);
    velocidade = aplicarPwmMinimo(velocidade);
    if (velocidade > 0) {
        ledcWrite(canalA, velocidade);
        ledcWrite(canalB, 0);
    }
    else if (velocidade < 0) {
        ledcWrite(canalA, 0);
        ledcWrite(canalB, -velocidade);
    }
    else {
        ledcWrite(canalA, 0);
        ledcWrite(canalB, 0);
    }
}
void pararSaidaMotores() {
    acionarMotor(CH_FE_A, CH_FE_B, 0);
    acionarMotor(CH_TE_A, CH_TE_B, 0);
    acionarMotor(CH_FD_A, CH_FD_B, 0);
    acionarMotor(CH_TD_A, CH_TD_B, 0);
}
void pararMovimentoImediato() {
    robot.targetVx = 0;
    robot.targetVy = 0;
    robot.targetVr = 0;
    robot.vx = 0;
    robot.vy = 0;
    robot.vr = 0;
    pararSaidaMotores();
}
void congelarServos() {
    // Garra permanece na ultima posicao comandada.
    // A cacamba para aproximadamente no ponto atual.
    robot.cacambaTarget = (uint8_t)constrain(
        (int)lroundf(robot.cacambaAtual),
        (int)ANGULO_CACAMBA_REPOUSO,
        (int)ANGULO_CACAMBA_ATIVA
    );
    // Reinicia o relogio da rampa para evitar um dt acumulado grande
    // quando a cacamba voltar a receber um novo alvo.
    robot.lastCacambaUpdate = millis();
}
void congelarAtuadoresParaSalvar() {
    pararMovimentoImediato();
    congelarServos();
}
void configurarMotores() {
    const uint8_t canais[] = {
        CH_FE_A, CH_FE_B,
        CH_TE_A, CH_TE_B,
        CH_FD_A, CH_FD_B,
        CH_TD_A, CH_TD_B
    };
    const uint8_t pinos[] = {
        PIN_FE_A, PIN_FE_B,
        PIN_TE_A, PIN_TE_B,
        PIN_FD_A, PIN_FD_B,
        PIN_TD_A, PIN_TD_B
    };
    for (int i = 0; i < 8; i++) {
        ledcSetup(canais[i], PWM_FREQ, PWM_BITS);
        ledcAttachPin(pinos[i], canais[i]);
    }
    pararMovimentoImediato();
}
// ============================================================
// JOYSTICK / TURBO / SUAVIZACAO
// ============================================================
int16_t mapearEixo(int valor, int maxSaida, bool inverter = false) {
    valor = constrain(valor, -512, 512);
    if (inverter) {
        valor = -valor;
    }
```

```
    int magnitude = abs(valor);
    if (magnitude <= DEADZONE) {
        return 0;
    }
    // Curva quadratica: mais precisao perto do centro sem perder
    // o comando maximo no fim do curso do analogico.
    float normalizado =
        (float)(magnitude - DEADZONE) / (512.0f - DEADZONE);
    float curva = normalizado * normalizado;
    int saida = (int)lroundf(curva * maxSaida);
    saida = constrain(saida, 0, maxSaida);
    return valor < 0 ? -saida : saida;
}
uint8_t escalaVelocidadeRT(ControllerPtr ctl) {
    int rt = constrain(ctl->throttle(), 0, 1023);
    float proporcao = rt / 1023.0f;
    float fator = VELOCIDADE_NORMAL +
        (VELOCIDADE_TURBO - VELOCIDADE_NORMAL) * proporcao;
    return (uint8_t)constrain(
        (int)lroundf(PWM_MAX * fator),
        (int)ESCALA_NORMAL,
        (int)ESCALA_TURBO
    );
}
bool joysticksNeutros(ControllerPtr ctl) {
    return
        abs(ctl->axisX()) <= DEADZONE &&
        abs(ctl->axisY()) <= DEADZONE &&
        abs(ctl->axisRX()) <= DEADZONE;
}
void atualizarMovimentoSuave() {
    uint32_t agora = millis();
    uint32_t dtMs = agora - robot.lastMovimentoUpdate;
    if (dtMs == 0) return;
    robot.lastMovimentoUpdate = agora;
    float alpha = 1.0f - expf(-(float)dtMs / TAU_MOVIMENTO_MS);
    alpha = constrain(alpha, 0.0f, 1.0f);
    robot.vx += (robot.targetVx - robot.vx) * alpha;
    robot.vy += (robot.targetVy - robot.vy) * alpha;
    robot.vr += (robot.targetVr - robot.vr) * alpha;
    if (fabsf(robot.vx) < 0.5f) robot.vx = 0;
    if (fabsf(robot.vy) < 0.5f) robot.vy = 0;
    if (fabsf(robot.vr) < 0.5f) robot.vr = 0;
}
// ============================================================================
// CINEMATICA MECANUM
// ============================================================================
void acionarMecanum() {
    float fe = robot.vy + robot.vx + robot.vr;
    float te = robot.vy - robot.vx + robot.vr;
    float fd = robot.vy - robot.vx - robot.vr;
    float td = robot.vy + robot.vx - robot.vr;
    // Primeiro preserva a proporcao entre as rodas.
    float maior = max(
        max(fabsf(fe), fabsf(te)),
        max(fabsf(fd), fabsf(td))
    );
    if (maior > PWM_MAX) {
        float k = (float)PWM_MAX / maior;
        fe *= k;
        te *= k;
        fd *= k;
        td *= k;
    }
    // Depois aplica o RT. Assim 70% e realmente o teto FINAL das rodas.
    float fatorSaida = robot.escalaSaida / (float)PWM_MAX;
    fe *= fatorSaida;
    te *= fatorSaida;
    fd *= fatorSaida;
    td *= fatorSaida;
    int8_t geral = INVERTER_SENTIDO_GERAL ? -1 : 1;
    acionarMotor(
        CH_FE_A,
        CH_FE_B,
        lroundf(fe) * SINAL_FE * geral
    );
    acionarMotor(
        CH_TE_A,
        CH_TE_B,
        lroundf(te) * SINAL_TE * geral
    );
    acionarMotor(
        CH_FD_A,
        CH_FD_B,
        lroundf(fd) * SINAL_FD * geral
    );
    acionarMotor(
        CH_TD_A,
        CH_TD_B,
        lroundf(td) * SINAL_TD * geral
    );
}
// ============================================================================
// APRENDIZADO DA MACRO
// ============================================================================
MacroStep capturarComandoAtual() {
    MacroStep cmd = {};
    cmd.duration = 0;
    cmd.vx = robot.targetVx;
    cmd.vy = robot.targetVy;
    cmd.vr = robot.targetVr;
    cmd.garra = robot.garraAtiva
        ? ANGULO_GARRA_ATIVA
        : ANGULO_GARRA_REPOUSO;
    cmd.cacamba = robot.cacambaTarget;
    cmd.escala = robot.escalaSaida;
    cmd.flags = FLAG_NENHUMA;
    return cmd;
}
bool transicaoCriticaEixo(int16_t anterior, int16_t atual) {
    // Parar/comecar e inverter sentido nunca podem ser filtrados como ruido.
    if ((anterior == 0) != (atual == 0)) return true;
    if (
          (anterior < 0 && atual > 0) ||
          (anterior > 0 && atual < 0)
    ) {
          return true;
    }
    return false;
}
bool transicaoCriticaMovimento(const MacroStep& a, const MacroStep& b) {
    return
        transicaoCriticaEixo(a.vx, b.vx) ||
        transicaoCriticaEixo(a.vy, b.vy) ||
            transicaoCriticaEixo(a.vr, b.vr);
}
bool comandoMudou(const MacroStep& a, const MacroStep& b) {
    if (transicaoCriticaMovimento(a, b)) return true;
    if (abs(a.vx - b.vx) >= DELTA_GRAVACAO) return true;
    if (abs(a.vy - b.vy) >= DELTA_GRAVACAO) return true;
    if (abs(a.vr - b.vr) >= DELTA_GRAVACAO) return true;
    if (a.garra != b.garra) return true;
    if (a.cacamba != b.cacamba) return true;
    if (a.flags != b.flags) return true;
    if (abs((int)a.escala - (int)b.escala) >= DELTA_RT_GRAVACAO) return true;
    return false;
}
bool mesmoComando(const MacroStep& a, const MacroStep& b) {
    return
        a.vx == b.vx &&
        a.vy == b.vy &&
        a.vr == b.vr &&
        a.garra == b.garra &&
        a.cacamba == b.cacamba &&
        a.escala == b.escala &&
        a.flags == b.flags;
}
uint32_t limitarMomentoAprendizado(uint32_t momento) {
    uint32_t decorrido = momento - inicioAprendizado;
    if (decorrido > TEMPO_MAX_AUTONOMO_MS) {
        decorrido = TEMPO_MAX_AUTONOMO_MS;
    }
    return inicioAprendizado + decorrido;
}
bool adicionarSegmento(MacroStep comando, uint32_t duracao) {
    if (duracao == 0 || tempoMacroGravado >= TEMPO_MAX_AUTONOMO_MS) {
        return false;
    }
    uint32_t restante = TEMPO_MAX_AUTONOMO_MS - tempoMacroGravado;
    duracao = min(duracao, restante);
    if (duracao == 0) return false;
    comando.duration = (uint16_t)duracao;
    if (
          macroGravacaoCount > 0 &&
          mesmoComando(
              macroGravacao[macroGravacaoCount - 1],
              comando
          )
    ) {
          macroGravacao[macroGravacaoCount - 1].duration += (uint16_t)duracao;
          tempoMacroGravado += duracao;
          return true;
    }
    if (macroGravacaoCount >= MAX_PASSOS_MACRO) {
        return false;
    }
    macroGravacao[macroGravacaoCount] = comando;
    macroGravacaoCount++;
    tempoMacroGravado += duracao;
    return true;
}
void fecharSegmentoAtual(uint32_t momento) {
    if (!segmentoAtivo) return;
    momento = limitarMomentoAprendizado(momento);
    uint32_t duracao = momento - inicioSegmento;
    adicionarSegmento(estadoSegmento, duracao);
    segmentoAtivo = false;
}
void iniciarAprendizado() {
    // O aprendizado sempre comeca de movimento parado.
    pararMovimentoImediato();
    aguardandoNeutroParaIniciarGravacao = false;
    // O neutro ja foi confirmado antes de entrar aqui.
    // Evita exigir uma segunda centralizacao logo apos iniciar a gravacao.
    aguardandoNeutroParaDirigir = false;
    macroGravacaoCount = 0;
    tempoMacroGravado = 0;
    pausaComboAprendizadoAtiva = false;
    inicioAprendizado = millis();
    inicioSegmento = inicioAprendizado;
    estadoSegmento = capturarComandoAtual();
    segmentoAtivo = true;
    modoAtual = APRENDENDO;
    Serial.println();
    Serial.println("========================================");
    Serial.println(">> MODO APRENDER INICIADO");
    Serial.println(">> Dirija normalmente. Maximo: 30 s.");
    Serial.println(">> L1 + R1 por 3 s novamente = salvar.");
    Serial.println("========================================");
}
void finalizarAprendizado(uint32_t momentoFim) {
    if (modoAtual != APRENDENDO) return;
    momentoFim = limitarMomentoAprendizado(momentoFim);
    // Nenhuma saida continua ativa durante a escrita NVS.
    congelarAtuadoresParaSalvar();
    if (segmentoAtivo) {
        uint32_t duracao = momentoFim - inicioSegmento;
        adicionarSegmento(estadoSegmento, duracao);
    }
    segmentoAtivo = false;
    pausaComboAprendizadoAtiva = false;
    if (macroGravacaoCount == 0) {
        Serial.println(">> Gravacao vazia. Macro antiga preservada.");
        aguardandoSoltarParaGravar = false;
        comboAprenderDisparado = (inicioComboAprender != 0);
        aguardandoNeutroParaDirigir = true;
        modoAtual = meuControle ? CONTROLE : PROCURANDO;
        return;
    }
    Serial.printf(">> Gravados %u passos. Salvando...\n", macroGravacaoCount);
    if (salvarMacroPersistente(macroGravacao, macroGravacaoCount)) {
        memcpy(
            macroAtiva,
            macroGravacao,
            macroGravacaoCount * sizeof(MacroStep)
        );
        macroAtivaCount = macroGravacaoCount;
        Serial.println(">> NOVA MACRO SALVA COM SUCESSO.");
    }
    else {
        Serial.println(">> ERRO AO SALVAR.");
        Serial.println(">> Macro anterior foi preservada.");
    }
    // O mesmo hold nao pode armar outra gravacao.
    aguardandoSoltarParaGravar = false;
    // So fica travado se o termino ocorreu enquanto L1 + R1 ainda
    // pertencem a um combo em andamento. Em auto-save, fica livre.
    comboAprenderDisparado = (inicioComboAprender != 0);
    // Evita voltar a andar se o joystick ainda estiver inclinado.
    aguardandoNeutroParaDirigir = true;
    modoAtual = meuControle ? CONTROLE : PROCURANDO;
}
void abortarAprendizado(const char* motivo) {
    if (modoAtual != APRENDENDO) return;
    macroGravacaoCount = 0;
    tempoMacroGravado = 0;
    segmentoAtivo = false;
    pausaComboAprendizadoAtiva = false;
    aguardandoNeutroParaIniciarGravacao = false;
    pararMovimentoImediato();
    congelarServos();
    aguardandoNeutroParaDirigir = true;
    modoAtual = meuControle ? CONTROLE : PROCURANDO;
    Serial.printf(">> Aprendizado abortado: %s\n", motivo);
    Serial.println(">> Macro anterior preservada.");
}
void registrarAprendizado() {
    if (modoAtual != APRENDENDO) return;
    uint32_t agora = millis();
    // A verificacao principal de 30 s ocorre no inicio do loop.
    // Esta redundancia protege a funcao caso ela seja chamada por outro caminho.
    if (agora - inicioAprendizado >= TEMPO_MAX_AUTONOMO_MS) {
        Serial.println(">> 30 s atingidos. Salvando automaticamente.");
        finalizarAprendizado(inicioAprendizado + TEMPO_MAX_AUTONOMO_MS);
        return;
    }
    MacroStep atual = capturarComandoAtual();
    if (!segmentoAtivo) {
        estadoSegmento = atual;
        inicioSegmento = agora;
        segmentoAtivo = true;
        return;
```

```
    }
    if (!comandoMudou(atual, estadoSegmento)) {
        return;
    }
    bool mudouCritico = transicaoCriticaMovimento(atual, estadoSegmento);
    bool mudouServo =
        atual.garra != estadoSegmento.garra ||
        atual.cacamba != estadoSegmento.cacamba;
    bool mudouFlags = atual.flags != estadoSegmento.flags;
    uint32_t duracao = agora - inicioSegmento;
    // Paradas/inversoes, servos e flags sao sempre registrados na hora.
    if (!mudouCritico && !mudouServo && !mudouFlags && duracao < SEGMENTO_MINIMO_MS) {
        return;
    }
    adicionarSegmento(estadoSegmento, duracao);
    estadoSegmento = atual;
    inicioSegmento = agora;
    if (macroGravacaoCount >= MAX_PASSOS_MACRO) {
        Serial.println(">> LIMITE DE PASSOS ATINGIDO. Salvando.");
        finalizarAprendizado(agora);
    }
}
bool comboAprenderPressionado(ControllerPtr ctl) {
    return ctl->l1() && ctl->r1();
}
void iniciarPausaComboAprendizado(uint32_t agora) {
    if (modoAtual != APRENDENDO || pausaComboAprendizadoAtiva) return;
    // Fecha o movimento real exatamente no instante em que o combo comecou.
    fecharSegmentoAtual(agora);
    // Durante a tentativa de salvar, o robo fica realmente parado.
    congelarAtuadoresParaSalvar();
    // Se o operador soltar antes de 3 s, esta pausa ZERO sera gravada.
    estadoSegmento = capturarComandoAtual();
    estadoSegmento.flags = FLAG_PARADA_IMEDIATA | FLAG_CONGELAR_CACAMBA;
    inicioSegmento = agora;
    segmentoAtivo = true;
    pausaComboAprendizadoAtiva = true;
}
void processarComboAprender(ControllerPtr ctl) {
    bool pressionados = comboAprenderPressionado(ctl);
    uint32_t agora = millis();
    if (pressionados) {
        // Combo tem prioridade sobre joystick, A e Y.
        if (inicioComboAprender == 0) {
            inicioComboAprender = agora;
            if (modoAtual == APRENDENDO) {
                iniciarPausaComboAprendizado(agora);
            }
            else if (modoAtual == CONTROLE) {
                congelarAtuadoresParaSalvar();
            }
        }
        else {
            // Continua congelado enquanto o combo esta pressionado.
            congelarAtuadoresParaSalvar();
        }
        if (comboAprenderDisparado) return;
        if (agora - inicioComboAprender < TEMPO_COMBO_APRENDER_MS) {
            return;
        }
        comboAprenderDisparado = true;
        if (modoAtual == CONTROLE) {
            aguardandoSoltarParaGravar = true;
            Serial.println(">> SISTEMA ARMADO. Solte L1 + R1 para gravar.");
            return;
        }
        if (modoAtual == APRENDENDO) {
            // Descarta a pausa de confirmacao: a macro termina quando o combo comecou.
            segmentoAtivo = false;
            pausaComboAprendizadoAtiva = false;
            finalizarAprendizado(inicioComboAprender);
            return;
        }
        return;
    }
    // Soltou os botoes.
    uint32_t inicioComboAnterior = inicioComboAprender;
    bool comboFoiConfirmado = comboAprenderDisparado;
    inicioComboAprender = 0;
    comboAprenderDisparado = false;
    if (aguardandoSoltarParaGravar) {
        aguardandoSoltarParaGravar = false;
        aguardandoNeutroParaIniciarGravacao = true;
        congelarAtuadoresParaSalvar();
        Serial.println(">> SISTEMA ARMADO. Centralize os analogicos para iniciar a
            gravacao.");
        return;
    }
    if (
        modoAtual == APRENDENDO &&
        pausaComboAprendizadoAtiva &&
        !comboFoiConfirmado &&
        inicioComboAnterior != 0
    ) {
        // Foi apenas um toque curto: grava a pausa real e continua.
        fecharSegmentoAtual(limitarMomentoAprendizado(agora));
        pausaComboAprendizadoAtiva = false;
    }
}
// ================================================================
// AUTONOMO
// ================================================================
void aplicarOrientacao(
    int16_t& vx,
    int16_t& vy,
    int16_t& vr,
    OrientacaoMacro orientacao
) {
    switch (orientacao) {
        case NORMAL:
            break;
            case ESPELHAR_DIREITA:
                vx = -vx;
                vr = -vr;
                break;
            case INVERTER:
                vy = -vy;
                break;
            case ESPELHAR_ESQUERDA:
                vx = -vx;
                vy = -vy;
                vr = -vr;
                break;
    }
}
void aplicarEventosEntradaPasso(const MacroStep& cmd) {
    // Eventos discretos devem ser preservados mesmo se um atraso fizer
    // o cronometro atravessar um ou mais passos inteiros.
    if (cmd.flags & FLAG_PARADA_IMEDIATA) {
        pararMovimentoImediato();
    }
    if (cmd.flags & FLAG_CONGELAR_CACAMBA) {
        congelarServos();
    }
    if (cmd.garra >= 0) {
        atualizarGarra(cmd.garra != 0);
    }
    if (!(cmd.flags & FLAG_CONGELAR_CACAMBA) && cmd.cacamba >= 0) {
        robot.cacambaTarget = constrain(
            cmd.cacamba,
            ANGULO_CACAMBA_REPOUSO,
            ANGULO_CACAMBA_ATIVA
        );
    }
}
void iniciarAutonomo(OrientacaoMacro orientacao) {
    if (macroAtivaCount == 0) {
        Serial.println(">> NENHUMA MACRO GRAVADA. Use L1 + R1 para aprender.");
        return;
    }
    pararMovimentoImediato();
    modoAtual = AUTONOMO;
    orientacaoAtual = orientacao;
    robot.currentStep = 0;
    robot.passoAplicado = 0xFFFF;
    robot.stepStartTime = millis();
    robot.inicioAutonomo = robot.stepStartTime;
    Serial.printf(">> AUTONOMO INICIADO - %u passos - limite 30 s\n", macroAtivaCount);
}
void finalizarAutonomo(const char* motivo) {
    pararMovimentoImediato();
    // O autonomo terminou: a cacamba nao continua se movendo depois do prazo.
    // A garra mantem a ultima posicao comandada.
    congelarServos();
    aguardandoNeutroParaDirigir = true;
    if (meuControle && meuControle->isConnected()) {
        modoAtual = CONTROLE;
    }
    else {
        modoAtual = PROCURANDO;
    }
    Serial.printf(">> AUTONOMO %s\n", motivo);
}
void cancelarAutonomo() {
    if (modoAtual != AUTONOMO) return;
    finalizarAutonomo("CANCELADO");
}
void gerenciarAutonomo() {
    if (modoAtual != AUTONOMO) return;
    uint32_t agora = millis();
    if (agora - robot.inicioAutonomo >= TEMPO_MAX_AUTONOMO_MS) {
        finalizarAutonomo("FINALIZADO - LIMITE DE 30 s");
        return;
    }
    // Mantem a macro sincronizada com o relogio.
    // Se algum passo inteiro tiver vencido por atraso do loop, seus eventos
    // discretos (flags/garra/cacamba) ainda sao consumidos antes de avancar.
    while (robot.currentStep < macroAtivaCount) {
        const MacroStep& cmdVencido = macroAtiva[robot.currentStep];
        if (agora - robot.stepStartTime < cmdVencido.duration) {
            break;
        }
        if (robot.passoAplicado != robot.currentStep) {
            aplicarEventosEntradaPasso(cmdVencido);
            robot.passoAplicado = robot.currentStep;
        }
        robot.stepStartTime += cmdVencido.duration;
        robot.currentStep++;
    }
    if (robot.currentStep >= macroAtivaCount) {
        finalizarAutonomo("FINALIZADO");
        return;
    }
    const MacroStep& cmd = macroAtiva[robot.currentStep];
    bool entrouNoPasso = robot.passoAplicado != robot.currentStep;
    if (entrouNoPasso) {
        robot.passoAplicado = robot.currentStep;
        aplicarEventosEntradaPasso(cmd);
    }
    int16_t vx = cmd.vx;
    int16_t vy = cmd.vy;
    int16_t vr = cmd.vr;
    aplicarOrientacao(vx, vy, vr, orientacaoAtual);
    robot.targetVx = vx;
    robot.targetVy = vy;
    robot.targetVr = vr;
    robot.escalaSaida = constrain(cmd.escala, ESCALA_NORMAL, ESCALA_TURBO);
}
// ================================================================
// CONEXAO DO CONTROLE
// ================================================================
void aoConectar(ControllerPtr ctl) {
    if (meuControle != nullptr) return;
    meuControle = ctl;
    robot.ultimoDado = millis();
    robot.failsafeAtivo = false;
    // Se reconectar durante o autonomo, a macro continua.
    if (modoAtual != AUTONOMO) {
        modoAtual = CONTROLE;
        aguardandoNeutroParaDirigir = true;
    }
    Serial.printf(">> CONTROLE CONECTADO: %s\n", ctl->getModelName().c_str());
    BP32.enableNewBluetoothConnections(false);
}
void aoDesconectar(ControllerPtr ctl) {
    if (ctl != meuControle) return;
    meuControle = nullptr;
    BP32.enableNewBluetoothConnections(true);
    // Cancela qualquer combo de aprendizado que tenha ficado pela metade.
    inicioComboAprender = 0;
    comboAprenderDisparado = false;
    aguardandoSoltarParaGravar = false;
    aguardandoNeutroParaIniciarGravacao = false;
    pausaComboAprendizadoAtiva = false;
    aguardandoNeutroParaDirigir = true;
    // No autonomo, perder o controle nao interfere na macro.
    // A trava do disparo permanece ate o fim do autonomo.
    if (modoAtual == AUTONOMO) {
        Serial.println(">> Controle desconectado. AUTONOMO CONTINUA.");
        return;
    }
    macroDisparoTravado = false;
    if (modoAtual == APRENDENDO) {
        macroGravacaoCount = 0;
        tempoMacroGravado = 0;
        segmentoAtivo = false;
        pausaComboAprendizadoAtiva = false;
        Serial.println(">> Aprendizado abortado. Macro antiga preservada.");
    }
    pararMovimentoImediato();
    congelarServos();
    modoAtual = PROCURANDO;
    Serial.println(">> CONTROLE DESCONECTADO - rodas paradas; servos mantidos.");
}
// ================================================================
// PROCESSAR GAMEPAD
// ================================================================
void processarGamepad(ControllerPtr ctl) {
    static uint32_t ultimoPrint = 0;
    if (DEBUG_SERIAL && millis() - ultimoPrint > 500) {
        ultimoPrint = millis();
        Serial.printf(
            "LX:%4d LY:%4d RX:%4d RT:%4d | A:%d Y:%d | MODO:%d\n",
            ctl->axisX(),
            ctl->axisY(),
            ctl->axisRX(),
            ctl->throttle(),
            ctl->a(),
            ctl->y(),
            modoAtual
        );
    }
```

```
    // Durante o autonomo o controle serve apenas para cancelar com B.
    if (modoAtual == AUTONOMO) {
        if (ctl->b()) {
            cancelarAutonomo();
        }
        return;
    }
    // B tambem funciona como "desistir" do processo de aprendizado.
    // Tem prioridade sobre L1+R1 para impedir um salvamento acidental.
    bool aprendizadoEmCurso =
        aguardandoSoltarParaGravar ||
        aguardandoNeutroParaIniciarGravacao ||
        modoAtual == APRENDENDO;
    if (ctl->b() && aprendizadoEmCurso) {
        bool comboAindaPressionado = comboAprenderPressionado(ctl);
        aguardandoSoltarParaGravar = false;
        aguardandoNeutroParaIniciarGravacao = false;
        pausaComboAprendizadoAtiva = false;
        inicioComboAprender = 0;
        // Se L1+R1 ainda estiverem fisicamente pressionados, mantenha a
        // trava ate soltarem para que o mesmo hold nao rearme a gravacao.
        comboAprenderDisparado = comboAindaPressionado;
        if (modoAtual == APRENDENDO) {
            abortarAprendizado("cancelado com B");
            // abortarAprendizado() limpa estados; restaura apenas a trava
            // necessaria quando L1+R1 ainda continuam pressionados.
            comboAprenderDisparado = comboAindaPressionado;
        }
        else {
            congelarAtuadoresParaSalvar();
            aguardandoNeutroParaDirigir = true;
            modoAtual = meuControle ? CONTROLE : PROCURANDO;
            Serial.println(">> Preparacao do aprendizado cancelada com B.");
            Serial.println(">> Macro anterior preservada.");
        }
        return;
    }
    bool comboAprender = comboAprenderPressionado(ctl);
    // O combo vem ANTES de joystick/servos: enquanto segura L1+R1,
    // nenhum novo comando de A, Y ou analogicos entra.
    processarComboAprender(ctl);
    if (comboAprender) {
        return;
    }
    // Depois de armar o aprendizado, a gravacao so comeca com os
    // tres analogicos no centro. Isso garante um inicio reproduzivel.
    if (aguardandoNeutroParaIniciarGravacao) {
        congelarAtuadoresParaSalvar();
        if (!joysticksNeutros(ctl)) {
            return;
        }
        aguardandoNeutroParaIniciarGravacao = false;
        iniciarAprendizado();
        return;
    }
    // Toda retomada manual exige uma passagem pelo neutro antes de dirigir.
    if (aguardandoNeutroParaDirigir) {
        congelarAtuadoresParaSalvar();
        if (!joysticksNeutros(ctl)) {
            return;
        }
        aguardandoNeutroParaDirigir = false;
        Serial.println(">> Analogicos no neutro. Controle manual liberado.");
    }
    // Os eixos sao sempre mapeados em 0..255.
    // O RT sera aplicado somente depois da mistura/normalizacao mecanum.
    robot.targetVy = mapearEixo(ctl->axisY(), PWM_MAX, true);
    robot.targetVx = mapearEixo(ctl->axisX(), PWM_MAX, false);
    robot.targetVr = mapearEixo(ctl->axisRX(), PWM_MAX, false);
    robot.escalaSaida = escalaVelocidadeRT(ctl);
    // Garra e cacamba continuam momentaneas e independentes.
    atualizarGarra(ctl->a());
    robot.cacambaTarget = ctl->y()
        ? ANGULO_CACAMBA_ATIVA
        : ANGULO_CACAMBA_REPOUSO;
    if (modoAtual == APRENDENDO) {
        registrarAprendizado();
        return;
    }
    if (modoAtual != CONTROLE) {
        return;
    }
    uint8_t dp = ctl->dpad();
    bool comboAutonomo = ctl->r1() && dp;
    // So destrava depois que R1 ou D-Pad forem soltos.
    if (!comboAutonomo) {
        macroDisparoTravado = false;
    }
    if (comboAutonomo && !macroDisparoTravado) {
        macroDisparoTravado = true;
        if (dp & DPAD_UP) {
            iniciarAutonomo(NORMAL);
            return;
        }
        if (dp & DPAD_RIGHT) {
            iniciarAutonomo(ESPELHAR_DIREITA);
            return;
        }
        if (dp & DPAD_DOWN) {
            iniciarAutonomo(INVERTER);
            return;
        }
        if (dp & DPAD_LEFT) {
            iniciarAutonomo(ESPELHAR_ESQUERDA);
            return;
        }
    }
}
// ================================================================
// FAIL-SAFE DO MANUAL
// ================================================================
void gerenciarFailsafeManual() {
    if (modoAtual == AUTONOMO) {
        return;
    }
    if (!meuControle || !meuControle->isConnected()) {
        return;
    }
    if (millis() - robot.ultimoDado <= FAILSAFE_MS) {
        return;
    }
    if (modoAtual == APRENDENDO) {
        abortarAprendizado("sem dados do controle");
    }
    // Cancela combos incompletos.
    inicioComboAprender = 0;
    comboAprenderDisparado = false;
    aguardandoSoltarParaGravar = false;
    aguardandoNeutroParaIniciarGravacao = false;
    pausaComboAprendizadoAtiva = false;
    macroDisparoTravado = false;
    // Continua zerando o movimento enquanto a comunicacao estiver parada.
    pararMovimentoImediato();
    congelarServos();
    aguardandoNeutroParaDirigir = true;
    if (!robot.failsafeAtivo) {
        robot.failsafeAtivo = true;
        Serial.println(">> FAIL-SAFE: 300 ms sem dados. Motores parados.");
    }
}
// ================================================================
```

```
// SETUP
// ================================================================
void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println();
    Serial.println("========================================");
    Serial.println(" ORION - CODIGO UNIFICADO V4.1.2 - ESP32-S3");
    Serial.println("========================================");
    configurarMotores();
    configurarServos();
    escreverServoGarra(ANGULO_GARRA_REPOUSO);
    escreverServoCacamba(ANGULO_CACAMBA_REPOUSO);
    robot.garraAtiva = false;
    robot.cacambaAtual = ANGULO_CACAMBA_REPOUSO;
    robot.cacambaTarget = ANGULO_CACAMBA_REPOUSO;
    robot.escalaSaida = ESCALA_NORMAL;
    robot.lastCacambaUpdate = millis();
    robot.lastMovimentoUpdate = millis();
    memoriaDisponivel = preferences.begin("orion", false);
    if (memoriaDisponivel) {
        carregarMacroInicial();
    }
    else {
        macroAtivaCount = 0;
        Serial.println(">> ERRO: NVS nao iniciou. Manual funciona, mas macro nao sera
            salva.");
    }
    BP32.setup(&aoConectar, &aoDesconectar);
    BP32.enableVirtualDevice(false);
    if (LIMPAR_PAREAMENTO_NO_BOOT) {
        BP32.forgetBluetoothKeys();
    }
    Serial.println();
    Serial.println("*** SISTEMA PRONTO - CORE 2.x ***");
    Serial.println("RT = teto final de 70% a 100% das rodas");
    Serial.println("L1 + R1 por 3 s = armar; solte e centralize analogicos para gravar");
    Serial.println("B = cancelar preparacao/aprendizado ou cancelar autonomo");
    Serial.println("R1 + D-Pad = executar macro uma vez por aperto");
    Serial.println("Autonomo: maximo 30 s; ao terminar, rodas e cacamba congelam");
    Serial.println("A macro permanece salva apos executar/cancelar/desligar");
}
// ================================================================
// LOOP
// ================================================================
void loop() {
    bool houveAtualizacao = BP32.update();
    bool controleValido =
        meuControle &&
        meuControle->isConnected() &&
        meuControle->isGamepad();
    // O limite de 30 s tem prioridade sobre qualquer novo comando.
    // Assim nenhum segmento pode ser fechado depois do prazo.
    if (
        modoAtual == APRENDENDO &&
        millis() - inicioAprendizado >= TEMPO_MAX_AUTONOMO_MS
    ) {
        Serial.println(">> 30 s atingidos. Salvando automaticamente.");
        finalizarAprendizado(inicioAprendizado + TEMPO_MAX_AUTONOMO_MS);
    }
    // So tratamos como dado novo quando Bluepad32 informa atualizacao.
    // Nao usamos hasData() para evitar bloquear controles que ja conectaram.
    if (controleValido && houveAtualizacao) {
        robot.ultimoDado = millis();
        robot.failsafeAtivo = false;
        processarGamepad(meuControle);
    }
    if (modoAtual == AUTONOMO) {
        gerenciarAutonomo();
    }
    else {
        gerenciarFailsafeManual();
    }
    atualizarMovimentoSuave();
    acionarMecanum();
    atualizarCacamba();
    delay(LOOP_MS);
}
```